# quallmer: A toolkit for qualitative analysis with large language models

3 April 2026


## Summary

*quallmer* is an R package that enables researchers to apply large language model (LLM)-assisted qualitative coding to texts, images, PDFs, audio, and tabular data while maintaining the rigorous standards of transparency and traceability essential to qualitative research. The package provides a complete workflow for: (1) flexibly defining comprehensive codebooks tailored to specific research questions and data types; (2) applying LLM-based analyses; (3) replicating analyses across different models and settings; (4) comparing results using established inter-rater reliability metrics and validating outputs against human-coded gold standards; and (5) documenting complete audit trails that capture the full decision history of the workflow, including model parameters, timestamps, and parent-child relationships across analyses. This audit trail functionality operationalizes the trustworthiness criteria established by Lincoln and Guba (1985), meeting their standards of credibility, transferability, dependability, and confirmability in LLM-assisted qualitative research. Built on the *ellmer* package (Wickham et al. 2025), *quallmer* supports a wide range of LLM providers—including OpenAI, Anthropic Claude, Google Gemini, Mistral, and local models via Ollama—giving researchers flexibility in their choice of models.


## Statement of need

Qualitative content analysis requires systematic coding of textual or visual material according to well-defined coding schemes (Krippendorff 2019). Researchers applying these methods face a fundamental challenge: manual coding is rigorous but does not scale to large corpora. LLMs offer unprecedented capacity to process qualitative data at scale, but their integration into rigorous research workflows is non-trivial.

Existing AI-assisted tools fall short in different ways. Proprietary qualitative data analysis software such as NVivo (2026), MAXQDA (2026), and QDAMiner (2026) have added AI-assisted coding features, but even the most transparent of these (QDAMiner, which allows researchers to select models and edit prompts) operates entirely through a graphical interface with no code-based workflow, no comprehensive audit trail, and no built-in reliability assessment framework. Researchers who instead write custom LLM code gain flexibility but must build substantial infrastructure from scratch—managing codebooks, tracking provenance, computing reliability metrics, and documenting workflows—diverting effort from substantive research questions (e.g., Fang, Garcia Bernardo, and Kesteren 2026). General-purpose R packages for LLM access (Wickham et al. 2025; Lin and Safi 2025; Gruber and Weber 2024) and text analysis (Benoit et al. 2018) do not address the specific methodological requirements of trustworthy qualitative research.

*quallmer* fills this gap. The package is designed for social scientists, qualitative researchers, and computational social scientists who need to apply LLM-assisted qualitative coding at scale while maintaining methodological rigor. Researchers can use *quallmer* without deep programming expertise, lowering barriers to adoption while preserving the transparency, reliability, and reproducibility that rigorous qualitative work demands.

## State of the field

The dominant tools for computer-assisted qualitative data analysis are commercial interactive software packages: NVivo (2026), MAXQDA (2026), and QDAMiner (2026) are widely used in social science research.

These tools offer graphical interfaces that make qualitative coding accessible, but their workflows are inherently interactive and non-reproducible: analyses cannot be executed as code and results cannot be straightforwardly replicated by other researchers. All three have added AI-assisted coding features at various points, but none fully addresses these fundamental limitations. In addition, none of these tools is open-source.

NVivo has offered machine-learning-based auto-coding since version 11 (2015), including sentiment detection, theme identification, and pattern-based coding. Of these, only pattern-based coding—which extends a researcher's own manual coding to new material—is customisable; sentiment and theme detection use fixed categories and noun-phrase frequency analysis respectively. NVivo 15 (2024) added a generative AI Assistant with more flexible capabilities, including AI-suggested subcodes within the researcher's existing framework, but these remain GUI-driven with no programmatic interface. MAXQDA introduced AI-assisted coding in 2024 (version 24.5), and does provide some transparency: each AI-coded segment is accompanied by an explanation of the coding rationale, and AI-generated codes are visually distinguished from manual codes. However, MAXQDA does not disclose which model or version powers its AI, and provides no structured audit log or mechanism for exact replication. QDAMiner 2025 takes a notably different approach, allowing researchers to select their own LLM provider and model, edit the prompts used for coding, and monitor token usage and costs per project—the most transparent AI implementation among major CAQDAS tools. Yet like its competitors, QDAMiner operates entirely through a graphical interface with no code-based workflow.

Lincoln and Guba (1985) established credibility, transferability, dependability, and confirmability as the essential trustworthiness criteria for rigorous qualitative research, broadly corresponding to internal validity, external validity, reliability, and objectivity in conventional inquiry (King, Keohane, and Verba 1994; Seale 1999). Meeting these standards requires detailed *audit trails* documenting analytical decisions, coding instructions, model parameters, and the rationale behind key choices. Lincoln and Guba (1985, 319–20) describe six features such trails should provide: raw data, data reduction products, data reconstruction products, process notes, information on intentions and dispositions, and instrument development information. None of the major CAQDAS tools provides a mechanism for generating such comprehensive audit trails. While MAXQDA and QDAMiner offer partial documentation of AI coding decisions, no tool records the full provenance chain—including model identifiers, parameter settings, timestamps, and parent-child relationships across analyses—needed for systematic replication. The stochastic nature of LLM outputs compounds this problem: neither NVivo nor MAXQDA discloses or allows researchers to lock the underlying model version, so re-running an analysis may produce different results for reasons outside the researcher's control.

Among R packages, *ellmer* (Wickham et al. 2025), *ollamar* (Lin and Safi 2025), and *rollama* (Gruber and Weber 2024) provide flexible LLM interfaces but offer no qualitative methodology support; *quanteda* (Benoit et al. 2018) supports quantitative text analysis but not AI-assisted qualitative coding. *quallmer* builds on *ellmer* for LLM access rather than reimplementing that layer. As open-source software, it allows full inspection of its implementation, and its unique scholarly contribution is the codebook, audit trail, and reliability assessment framework that operationalizes Lincoln and Guba (1985)'s trustworthiness criteria directly in code.

## Software design

*quallmer* implements a structured five-step workflow through five core functions, summarised in **??**. Each function addresses specific requirements of trustworthy qualitative research as established by Lincoln and Guba (1985) and Krippendorff (2019).

Table 1: Core *quallmer* functions and their role in the five-step workflow.

| Step | Function(s) | Purpose |
|---|---|---|
| 1. Define codebook | `qlm_codebook()` | Create reusable coding schemes with instructions and structured output schema |

| Step | Function(s) | Purpose |
|---|---|---|
| 2. Code data | `qlm_code()`, `qlm_segment()` | Apply a codebook to texts, images, PDFs, audio, or tabular data using any supported LLM; or segment texts into thematic units and optionally apply codes in the same pass |
| 3. Replicate | `qlm_replicate()` | Re-run coding with different models or parameter settings, preserving provenance chains |
| 4. Compare & validate | `qlm_compare()`, `qlm_validate()` | Compute inter-rater reliability metrics; benchmark against human-coded gold standards |
| 5. Audit trail | `qlm_trail()` | Generate complete audit documentation and an executable replication report |

The package is built on the *ellmer* framework (Wickham et al. 2025), which provides a flexible interface to a wide range of LLM providers. This design ensures researchers can choose models that fit their research questions and resource constraints without *quallmer* managing LLM connectivity. The architecture is modular and extensible, allowing customization of codebooks, coding procedures, and validation metrics while ensuring all analyses are fully documented.

*quallmer* evolved from an earlier implementation (a draft R package entitled *quanteda.llm* (Maerz and Benoit 2025)) that integrated LLM capabilities with the *quanteda* framework (Benoit et al. 2018). Feedback from workshops and training sessions with social scientists shaped key design decisions: the highly customizable codebook definition, the built-in reliability metrics, and especially the audit trail functionality. A companion package, *quallmer.app* (also on CRAN), provides a Shiny application for manual coding, review of LLM-assisted coding, and agreement metric computation, bridging automated and human-in-the-loop workflows.

For analyses requiring variable-length text units, `qlm_segment()` uses an LLM to segment texts into thematic or conceptual units—useful for aspect-based sentiment analysis, quasi-sentence segmentation, or topic-based chunking. Codes can be applied to each segment in the same pass by including them in the codebook schema, making a separate `qlm_code()` call unnecessary. When comparing segmented outputs, `qlm_compare()` computes Krippendorff's alpha for unitizing (Krippendorff 2019, sec. 12.6), which jointly measures agreement on both where boundaries fall and how segments are coded. The function also reports unitization-conditional alpha, isolating coding disagreement from boundary disagreement, as well as per-code reliability metrics for diagnosing which categories are applied consistently.

## Worked example: coding political speeches

To illustrate the workflow, consider analyzing political rhetoric in speeches. The complete tutorial is at https://quallmer.github.io/quallmer/.

**Step 1: Codebook definition.** `qlm_codebook()` defines a reusable coding scheme specifying instructions for the LLM and the expected output structure using *ellmer* type specifications:

```
library(quallmer)

codebook_ideology <- qlm_codebook(
  name = "Liberal-illiberal rhetoric",
  instructions = "Analyze the rhetorical style of this political speech.
    ILLIBERAL rhetoric (negative scores): nationalism, paternalism, traditionalism.
    LIBERAL rhetoric (positive scores): individual rights, tolerance, civil liberties.
    Score 0 = neutral/mixed.",
```

```
  schema = type_object(
    score = type_integer("Score from -10 (illiberal) to +10 (liberal)"),
    explanation = type_string("Brief explanation of the score")
  )
)
```

**Step 2: Data coding.** `qlm_code()` applies the codebook to texts using any *ellmer*-supported LLM, returning a `qlm_coded` object with coding results and full metadata (timestamps, model identifiers, codebook used):

```
coded_speeches <- qlm_code(
  speech_corpus,                  # character vector or quanteda corpus
  codebook = codebook_ideology,   # qlm_codebook object
  model = "openai/gpt-4o-mini",   # provider and model
  name = "ideology_coding_run1"   # user-assigned model name
)
```

**Step 3: Replication.** `qlm_replicate()` re-runs coding with different models or parameter settings, maintaining provenance chains linking replicated results to their parent analyses:

```
coded_claude  <- qlm_replicate(coded_speeches, model = "anthropic/claude-3-5-sonnet")
coded_temp07  <- qlm_replicate(coded_speeches, temperature = 0.7)
```

**Step 4: Comparison and validation.** `qlm_compare()` computes inter-rater reliability tailored to measurement level: Krippendorff's alpha, Cohen's/Fleiss' kappa, and percent agreement for nominal data; Krippendorff's alpha, weighted kappa, Kendall's W, Spearman's rho, and percent agreement for ordinal data; and Krippendorff's alpha, ICC, Pearson's r, and percent agreement for interval/ratio data. For segmented corpora, it additionally computes Krippendorff's unitizing alpha (boundary agreement), unitizing alpha with codes (boundary and coding agreement jointly), coding-conditional alpha (coding agreement given shared boundaries), and per-category unitizing alpha. `qlm_validate()` benchmarks against human-coded gold standards, computing accuracy, precision, recall, F1, and Cohen's kappa for nominal data; Spearman's rho, Kendall's tau, and MAE for ordinal data; and ICC, Pearson's r, MAE, and RMSE for interval data:

```
comparison <- qlm_compare(coded_speeches, coded_claude, by = "score", level = "interval")
validation <- qlm_validate(coded_speeches, gold_standard, by = "score", level = "interval")
```

**Step 5: Audit trail.** `qlm_trail()` generates a complete audit package following Lincoln and Guba (1985), including an `.rds` trail object and an executable Quarto replication report:

```
qlm_trail(coded_speeches, path = "replication_materials")
```

## Research impact statement

*quallmer* is available on CRAN and actively maintained at https://github.com/quallmer/quallmer. The companion *quallmer.app* package is also on CRAN. Applications have included large-scale content analysis of political documents, media coverage, and sentiment analysis among others. The package has been tested and applied in ongoing research projects by the authors and collaborating researchers (including Andrew Walter and Dean Schafer), with peer-reviewed publications using *quallmer* in preparation. Its theoretical grounding in Lincoln and Guba (1985)'s trustworthiness framework, combined with integration into the established *ellmer* and *quanteda* R ecosystems, positions *quallmer* as methodological infrastructure for the growing community of computational social science researchers applying LLMs to qualitative data.

## AI usage disclosure

In developing *quallmer*, we used AI coding assistants, specifically Anthropic's Claude (via Claude Code) and GitHub Copilot, for code refactoring, documentation drafting, and test development. All AI-generated contributions were reviewed, validated, and revised by the human authors, who retained full responsibility for all design decisions, the intellectual framing of the package, and the conceptual framework grounding the audit trail functionality in established qualitative methodology. The use of AI tools accelerated implementation but did not substitute for the substantive scholarly decisions that define the package's contribution. This paper was drafted with AI assistance; all content was reviewed and substantially revised by the authors.

## Acknowledgements

We gratefully acknowledge feedback from participants in our workshops and training sessions, whose experiences and suggestions substantially shaped the package design. We also thank Andrew Walter and Dean Schafer for testing early versions of the package, including *quallmer.app*.

## References

Benoit, Kenneth, Kohei Watanabe, Haiyan Wang, Paul Nulty, Adam Obeng, Stefan Müller, and Akitaka Matsuo. 2018. "Quanteda: An r Package for the Quantitative Analysis of Textual Data." *Journal of Open Source Software* 3 (30): 774. https://doi.org/10.21105/joss.00774.

Fang, Qixiang, Javier Garcia Bernardo, and Erik-Jan van Kesteren. 2026. "A Methodological Guide on Using Large Language Models for Text Annotation in the Social Sciences and Humanities with Python and R." OSF preprint. https://osf.io/preprints/socarxiv/v4eq6_v2.

Gruber, Johannes B., and Maximilian Weber. 2024. "rollama: An R package for using generative large language models through Ollama," 1–4. http://arxiv.org/abs/2404.07654.

King, Gary, Robert O. Keohane, and Sidney Verba. 1994. *Designing Social Inquiry: Scientific Inference in Qualitative Research*. Princeton, NJ: Princeton University Press.

Krippendorff, Klaus. 2019. *Content Analysis: An Introduction to Its Methodology*. 4th ed. Thousand Oaks, CA: SAGE Publications.

Lin, Hause, and Tawab Safi. 2025. "ollamar: An R package for running large language models." *Journal of Open Source Software* 10 (105): 7211. https://doi.org/10.21105/joss.07211.

Lincoln, Yvonna S., and Egon G. Guba. 1985. *Naturalistic Inquiry*. Beverly Hills, CA: SAGE Publications.

Maerz, Seraphine F., and Kenneth Benoit. 2025. *Quanteda.llm: LLM Support for Quanteda*. https://github.com/quanteda/quanteda.llm.

MAXQDA. 2026. "Discover What Is Possible with AI Assist." https://www.maxqda.com/products/ai-assist.

NVivo. 2026. "NVivo Automated Coding with AI." https://lumivero.com/resources/product-tutorials/nvivo-automated-coding-with-ai/.

Provalis Research. 2026. "QDAMiner Qualitative Data Analysis Software." https://provalisresearch.com/products/qualitative-data-analysis-software/.

Seale, Clive. 1999. "Guiding Ideals." In *The Quality of Qualitative Research*, 32–50. London: SAGE Publications.

Wickham, Hadley, Joe Cheng, Aaron Jacobs, Garrick Aden-Buie, and Barret Schloerke. 2025. *Ellmer: Chat with Large Language Models*. https://ellmer.tidyverse.org.